

🔐 Bộ Câu Hỏi Ôn Tập — Mini EASM

Đối tượng: Người làm Cybersecurity đang học hiểu code + phát triển + bảo mật ứng dụng. **Định hướng:** Câu hỏi đánh vào **ngữ cảnh, solution, mindset** — không hỏi lý thuyết suông, không leetcode. **Tỷ lệ:** ~20% kiến thức dev tổng quan · ~80% bám vào code thật của project.

Cách dùng: Trả lời bằng lời của bạn, mở file gợi ý (🔐) ra **đối chiếu**. Mỗi câu nên trả lời được 3 ý: (1) *Code đang làm gì?* (2) *Tại sao làm vậy?* (3) *Nếu là attacker / nếu đi production thì sao?*

🔐 BỘ 1 — Hiểu Code, Pipeline & Workflow

(Mục tiêu: đọc được luồng dữ liệu, biết hàm nào gọi hàm nào, vì sao tách lớp)

A. Kiến thức dev tổng quan (20%)

1.1. Project chia 5 lớp `route → handler → service → repository → DB`. Hãy giải thích **mỗi lớp được phép làm gì và bị cấm làm gì**. Tại sao việc "chỉ repository mới được viết SQL" lại là một quyết định vừa về *bảo trì* vừa về *bảo mật*?

🔐 `backend/src/routes/index.js, handlers/, services/, repositories/`

1.2. Endpoint scan trả về **202** ngay lập tức rồi chạy nền (fire-and-forget). So với việc xử lý đồng bộ rồi trả 200, pattern này **đánh đổi điều gì?** Trong ngữ cảnh một công cụ quét bảo mật (scan có thể chạy lâu), tại sao 202 lại hợp lý?

🔐 `services/scan.service.js (startScan, setImmediate)`

1.3. Project dùng `better-sqlite3` với **prepared statement** (`db.prepare('... WHERE id = ?').get(id)`) thay vì nối chuỗi SQL. Giải thích vì sao cách này chặn được SQL Injection ở tầng gốc, và vì sao "dùng ORM" không phải là điều kiện bắt buộc để an toàn.

🔐 `repositories/asset.repository.js`

1.4. Database bật `journal_mode = WAL` và `foreign_keys = ON` mỗi khi mở kết nối. `foreign_keys = ON` ảnh hưởng thế nào tới hành vi `ON DELETE CASCADE` khi xóa một asset? Điều gì xảy ra với `scan_jobs / scan_results` của asset đó?

🔐 `db/connection.js, db/migrations/*.sql`

B. Đọc luồng & pipeline (code)

1.5. **Trace toàn bộ vòng đời một lần scan**: từ lúc user bấm nút trên UI → tới khi kết quả hiện ra màn hình. Liệt kê tuần tự các hàm/file đi qua (frontend `api.js` → `route → handler → service → repository → poll`).

🔐 `frontend/js/api.js, routes/index.js, handlers/scan.handler.js, services/scan.service.js`

1.6. Trong `scan.service.js`, hàm `startScan` **return job rồi mới** `setImmediate(() => runScanAsync(...))`. Giải thích vì sao thứ tự này quan trọng — nếu đảo lại (chạy scan trước, return sau) thì hành vi HTTP thay đổi ra sao?

1.7. Trạng thái một job đi qua `pending → running → completed/failed/partial`. Dựa vào `runScanAsync`, hãy nói **chính xác** khi nào job thành `completed`, khi nào `partial`, khi nào `failed`. Vì sao `scan_type='all'` lại có thể ra `partial` còn scan đơn lẻ thì không?

🔐 `services/scan.service.js` (vòng lặp `Promise.allSettled`)

1.8. Route khai báo `getLatestResult('dns'), getLatestResult('whois')` ... Đây là kỹ thuật gì (hàm trả về hàm)? Giải thích cách một handler được "cấu hình" sẵn `scanType` trước khi Express gọi nó với (`req, res, next`).

🔐 `handlers/scan.handler.js, routes/index.js`

1.9. Trong `asset.handler.js`, `create` là một **mảng** [`validate(schema), asyncFn`] còn `list` chỉ là một hàm. Express xử lý mảng middleware này thế nào? Tại sao `validate` phải nằm **trước** handler chính?

🔐 `handlers/asset.handler.js, middleware/validate.js`

1.10. `middleware/validate.js` có dòng `req.body = result.data;` — gán đè body bằng dữ liệu đã qua Zod. Vì sao việc **thay thế** body (thay vì chỉ kiểm tra rồi đi tiếp) lại quan trọng về mặt bảo mật? (Gợi ý: field thừa client gửi lên sẽ ra sao?)

1.11. `getDb()` dùng pattern **singleton lazy** (`if (!db) db = new Database(...)`). Lợi ích là gì? Nếu mỗi truy vấn đều mở kết nối mới thì pragma `foreign_keys = ON` có còn tác dụng không?

🔐 `db/connection.js`

1.12. Migration runner đọc các file `.sql`, sort theo tên, bỏ qua file đã có trong bảng `_migrations`, và chạy **mỗi file trong 1 transaction**. Nếu một file SQL chạy lỗi giữa chừng, điều gì xảy ra với các câu lệnh trước đó trong cùng file? Vì sao đặt tên file dạng `001_`, `002_` lại quan trọng?

🔐 `db/migrate.js`

1.13. DNS scanner nhận `run(asset, { resolver = dnsPromises } = {})` — cho phép **tiêm (inject) resolver**. Mục đích thiết kế này là gì? Nó giúp việc viết unit test (không cần internet thật) như thế nào?

🔐 `scanners/dns.scanner.js, CLAUDE.md` mục Testing

1.14. `app.js` mount API router **trước** rồi mới `express.static(frontend)`. Nếu đảo thứ tự (static trước, router sau) thì điều gì có thể hỏng? Vì sao thứ tự đăng ký middleware trong Express lại có ý nghĩa?

```
❏ app.js
```

1.15. Frontend `jobs.js` không có endpoint "list tất cả jobs" — nó **lập qua từng asset** rồi gọi `getAssetScans(a.id)` và gộp lại (`Promise.all + flat`). Đây là hệ quả của thiết kế API ở backend. Hãy chỉ ra điểm yếu về hiệu năng/khả năng mở rộng của cách làm này khi có hàng nghìn asset (vấn đề N+1).

```
❏ frontend/js/jobs.js, routes/index.js
```

1.16. `error.js` chỉ `logger.error` khi `status >= 500`, còn lỗi 4xx thì không log err object. Giải thích lý do (phân biệt "lỗi do client" vs "lỗi do server"). Response lỗi luôn có field `details` — field này lấy từ đâu khi validate thất bại?

```
❏ middleware/error.js, middleware/validate.js, utils/errors.js
```

1.17. Scanner registry (`scanners/index.js`) build một map `type → scanner` bằng `Object.fromEntries`. Khi service nhận `scan_type='ssl'`, nó tra cứu scanner ra sao? `'all'` **không phải** một scanner trong map — nó được xử lý ở đâu và bằng cơ chế gì?

```
❏ scanners/index.js, services/scan.service.js
```

1.18. Mỗi scanner export `appliesTo: ['domain']` hoặc `['ip']`. Trước khi chạy, service kiểm tra `scanner.appliesTo.includes(asset.type)`. Hãy giải thích vì sao một asset kiểu `domain` không thể chạy `port scan`, và lỗi đó được trả về client với HTTP status nào?

```
❏ services/scan.service.js, scanners/port.scanner.js
```

❏ BỘ 2 — Mindset Bảo Mật & Threat Modeling

(Mục tiêu: nhìn code như một attacker, đánh giá rủi ro, đề xuất solution phòng thủ)

A. Kiến thức dev/security tổng quan (20%)

2.1. Toàn bộ API **không có authentication/authorization**. Trong ngữ cảnh đây là công cụ *chủ động đi quét bên thứ ba* (DNS, port, SSL, fetch HTTP...), việc để API mở khiến nó có thể bị biến thành thứ gì? (Gợi ý: "scan proxy" / công cụ tấn công gián tiếp). Bạn sẽ thêm lớp phòng thủ nào đầu tiên?

2.2. CORS được set `Access-Control-Allow-Origin: *`. Giải thích khi nào `*` là chấp nhận được và khi nào là rủi ro. Nếu sau này API có thêm cookie/session, vì sao `*` kết hợp với credentials lại trở thành lỗ hổng?

```
❏ middleware/cors.js
```

2.3. Pipeline CI có `npm audit`, `Trivy`, `Gitleaks`, `TruffleHog`. Hãy phân biệt **mỗi tool bắt loại rủi ro nào** (lỗ hổng dependency / lỗ hổng image-OS / secret bị commit). Vì sao có cả `Gitleaks` lẫn `TruffleHog` — chúng khác nhau ở điểm gì?

```
❏ .github/workflows/ci.yml
```

2.4. Đề bài gốc yêu cầu `Gosec` (cho Go) nhưng project là Node.js nên thay bằng `eslint-plugin-security + npm audit`. Đây là ví dụ về "chọn công cụ theo ngữ cảnh stack". Giải thích vì sao chạy một SAST tool sai ngôn ngữ là vô nghĩa, và `eslint-plugin-security` thực chất quét được loại vấn đề gì trong JS.

B. Threat modeling trên code thật

2.5. **(SSRF — câu trọng tâm)** Schema tạo asset dùng regex `^[a-zA-Z0-9._-:]+$` cho `name`. Regex này **cho phép cả dấu : và /**. `tech.scanner.js` sau đó `fetch('https://' + asset.name)`. Hãy chỉ ra: một attacker có thể nhập `name` như thế nào để biến scanner thành công cụ **SSRF** đánh vào dịch vụ nội bộ (vd metadata endpoint `169.254.169.254`, hay `localhost:8080/...`)? Bạn sẽ siết validation ra sao?

```
❏ models/asset.js, scanners/tech.scanner.js, scanners/ip.scanner.js
```

2.6. **(Port scan safety)** `port.scanner.js` chặn bằng `isPrivateIP(ip)` — chỉ cho quét IP private. Đọc kỹ `utils/safety.js`: hàm check **chuỗi `asset.name` trực tiếp**. Câu hỏi: (a) Vì sao hạn chế "chỉ private IP" lại là vấn đề *pháp lý/đạo đức*, không chỉ kỹ thuật? (b) Logic hiện tại có sơ hở nào với input dạng tên miền phân giải ra IP nội bộ (DNS rebinding), hay nó an toàn vì chỉ nhận IP literal? Lập luận.

```
❏ scanners/port.scanner.js, utils/safety.js
```

2.7. `ssl.scanner.js` đặt `rejectUnauthorized: false` khi `tls.connect`. (a) Vì sao trong ngữ cảnh **recon/thu thập thông tin cert**, việc này lại *cần thiết* (cert hết hạn / self-signed vẫn phải đọc được)? (b) Nhưng nếu cùng đoạn code đó được copy sang một client gọi API thật trong production thì nó trở thành lỗ hổng gì (MITM)? Đây là bài học về "đúng/sai tùy ngữ cảnh".

```
❏ scanners/ssl.scanner.js
```

2.8. **(Stored XSS / data poisoning)** `subdomain.scanner.js` tin tưởng hoàn toàn JSON từ `crt.sh`, lấy `name_value` của cert và lưu thẳng vào DB. Frontend sau đó render kết quả. (a) Một bên thứ ba kiểm soát nội dung cert (CN/SAN) có thể nhồi payload vào không? (b) Tầng nào đang bảo vệ frontend khỏi việc render payload đó? Tìm hàm phòng thủ trong `frontend/js/ui.js`.

```
❏ scanners/subdomain.scanner.js, frontend/js/jobs.js (dùng escapeHtml), frontend/js/ui.js
```

2.9. **(DoS / resource exhaustion)** `runScanAsync` gọi `scanner.run(asset)` nhưng **không có timeout tổng cho cả job**. Nếu một scanner (vd `whois`, `tech`) treo do mạng, job sẽ kẹt ở trạng thái nào mãi mãi? Kết hợp với việc *không có rate limit* và *không có auth*, hãy mô tả một kịch bản DoS nhắm vào chính server này hoặc vào dịch vụ bên thứ ba (`crt.sh`, `ip-api.com`).

```
❏ services/scan.service.js, scanners/tech.scanner.js, scanners/whois.scanner.js
```

2.10. `tech.scanner.js`: nếu fetch HTTPS lỗi thì **fallback sang HTTP** (`http://`). Phân tích rủi ro của việc tự động hạ cấp xuống HTTP trong một công cụ bảo mật: thông tin gì có thể bị lộ/giả mạo trên đường truyền? Có nên fallback im lặng hay phải đánh dấu rõ kết quả "thu được qua HTTP không mã hóa"?

```
❏ scanners/tech.scanner.js
```

2.11. DNS scanner bực **mỗi** lần resolve trong `try/catch` rồi trả `[]` khi lỗi, thay vì để throw. Đây là mindset "graceful degradation". (a) Lợi ích cho UX/robustness là gì? (b) Nhưng về mặt *bảo mật/độ tin cậy của dữ liệu* việc "nuốt lỗi" có nguy cơ gì — làm sao phân biệt "domain không có record MX" với "query MX bị lỗi mạng"? Cả hai đều ra `[]`.

📄 `scanners/dns.scanner.js`

2.12. `scan_results.data` lưu nguyên `JSON.stringify(results)`. `whois` scanner trả về object WHOIS có thể rất lớn. (a) Có giới hạn kích thước nào không? (b) Một asset độc hại trả về response khổng lồ (vd server WHOIS/HTTP cố tình trả vài chục MB) có thể gây vấn đề gì cho SQLite/bộ nhớ? Đề xuất biện pháp giới hạn.

📄 `services/scan.service.js`, `scanners/whois.scanner.js`, `scanners/tech.scanner.js` (`res.text()`)

2.13. `error.js` trả về `message: err.message` cho client kể cả lỗi 500 (chỉ ẩn `err` object khỏi *log*, không ẩn khỏi *response*). Trong threat modeling, vì sao **thông điệp lỗi chi tiết** (`stack`, `path`, lỗi DB...) lại là rủi ro *information disclosure*? Một lỗi 500 nên trả gì cho client ở production?

📄 `middleware/error.js`

2.14. CI `trivy filesystem` scan dùng `skip-dirs: 'node_modules'`, trong khi `job audit` lại chạy `npm audit` (vốn xét chính dependencies). Giải thích vì sao đây *không* phải lỗ hổng phủ sót — hai job bù cho nhau ra sao? Nếu cả hai cùng bỏ qua dependency thì rủi ro gì lọt lưới?

📄 `.github/workflows/ci.yml`

2.15. TruffleHog chạy với `--only-verified` và Gitleaks với `--exit-code=1`. (a) `--only-verified` đánh đổi điều gì (false positive ↓ nhưng false negative ↑)? (b) Trong ngữ cảnh "secret đã bị commit vào git history", vì sao cả hai job đều cần `fetch-depth: 0` (full history) mới quét đúng?

📄 `.github/workflows/ci.yml`

2.16. `build-images` job chỉ chạy if: `github.event_name == 'pull_request'` và needs: `[lint, test]`. Giải thích chiến lược "gate": vì sao không build/scan image trên *mọi* push, và vì sao build image phải **đợi** lint + test xanh trước? Liên hệ tới khái niệm "fail fast / shift-left security".

📄 `.github/workflows/ci.yml`

2.17. Frontend `api.js` khi gặp lỗi sẽ `throw new Error(data?.error?.message || ...)` — tức là **hiển thị thẳng message từ server** cho người dùng (qua Toast). Kết hợp với câu 2.13, hãy phân tích chuỗi rủi ro: server leak chi tiết lỗi → client render. Nếu message chứa dữ liệu do attacker kiểm soát, có thành XSS không? Toast render bằng `textContent` hay `innerHTML`?

📄 `frontend/js/api.js`, `frontend/js/ui.js` (xem cách `Toast` render)

2.18. Tổng kết — **Threat model 1 trang**: Liệt kê 5 **lỗ hổng/rủi ro nghiêm trọng nhất** của Mini EASM nếu đem deploy public *nguyên trạng* (gợi ý: thiếu auth, SSRF qua scanner, không rate limit, info disclosure, app tự đi tấn công bên thứ 3). Với mỗi cái: xếp hạng mức độ, và đề xuất 1 **biện pháp khắc phục cụ thể** (nêu rõ sửa ở file/lớp nào trong kiến trúc 5 lớp).

📄 Cách tự chấm

Với mỗi câu, bạn nắm chắc khi trả lời được:

1. **Code làm gì** — đọc được luồng, chỉ đúng file/hàm.
2. **Tại sao thiết kế vậy** — hiểu đánh đổi (trade-off), không học vẹt.
3. **Góc nhìn attacker/production** — rủi ro là gì, và ở đâu.

📄 Các câu gần 📄 là **trọng tâm bảo mật** — nếu chỉ có thời gian ôn ít, ưu tiên: 2.5 (SSRF), 2.6 (port scan safety), 2.8 (XSS/data poisoning), 2.18 (threat model tổng).